Jaypee Institute of Information Technology

Sector 62, Noida



Cloud and Web Services Lab [17M15CS121]

Project Report

Title: "Cloud-Based Medical Appointment System"

Submitted To:

Dr. P. Raghu Vamsi
Assistant Professor(Senior Grade) , Dept. of CSE
JIIT Sector 62, Noida

TEAM MEMBERS

Name	Enrollment No.	Batch
Sabeeh Ahsan	22803006	B12
Harleen Kaur	22803009	B12
Aryan Khanna	22803028	B12

PROJECT OUTLINE

S. No.	Title	Page No.
1	Abstract	3
2	Problem Statement	3
3	Objectives	4
4	System Architecture & Flowchart	5
5	Implementation Details	6
6	Database Design & API Endpoints	8
7	Cloud Deployment & Security	9
8	Tools & Technologies Used	10
9	Results & Outcomes	10
10	Conclusion & Future Scope	11
11	References	12

1. Abstract

The management of doctor appointments and patient records is still handled manually in a large number of clinics and small hospitals. This leads to overlapping bookings, misplaced prescriptions and long waiting queues. MediConnect is a cloud-hosted web application that digitises this entire workflow. It lets patients discover doctors by specialisation, reserve a free time slot in real time, and later revisit their consultation history, while doctors manage their own schedule and issue digital prescriptions that are stored in the cloud.

The application is built on the MERN stack – MongoDB for data, Express and Node.js for the REST API, and React (bundled with Vite) for the user interface. Instead of a single machine, the system is deployed on Amazon Web Services using a two-tier server layout: a public web server that faces the internet and a private database server that is reachable only from inside the virtual network. Prescription files are kept in an Amazon S3 bucket and served back through short-lived signed links. This report describes the problem being solved, the design of the system, the concrete implementation of each module, and the cloud infrastructure that makes the platform secure and reachable from any device with a browser.

2. Problem Statement

Booking a medical appointment through a phone call or a walk-in visit is inefficient for both sides of the counter. Patients have no visibility into which slots are genuinely free, so they are often turned away or asked to wait. Clinics maintain appointment diaries and paper prescriptions that are easy to lose, hard to search, and impossible to access remotely. When a patient needs an old prescription, retrieving it means physically returning to the clinic.

The core problems this project targets are:

- **No live availability –** there is no shared, real-time view of a doctor's availability, which causes double-booking of the same slot;
- **Paper records –** prescriptions and reports exist only on paper and are neither backed up nor accessible online;
- **Weak access control –** patient and doctor identities are not verified, so anyone could view or alter another person's data;
- **No remote access –** an on-premise system tied to one computer cannot be reached outside the clinic and fails if that machine goes down.

MediConnect answers these gaps with a single web platform that keeps one authoritative copy of every appointment, authenticates each user before granting access, stores prescriptions durably in the cloud, and remains available over the public internet.

3. Objectives

The project was carried out with the following concrete objectives:

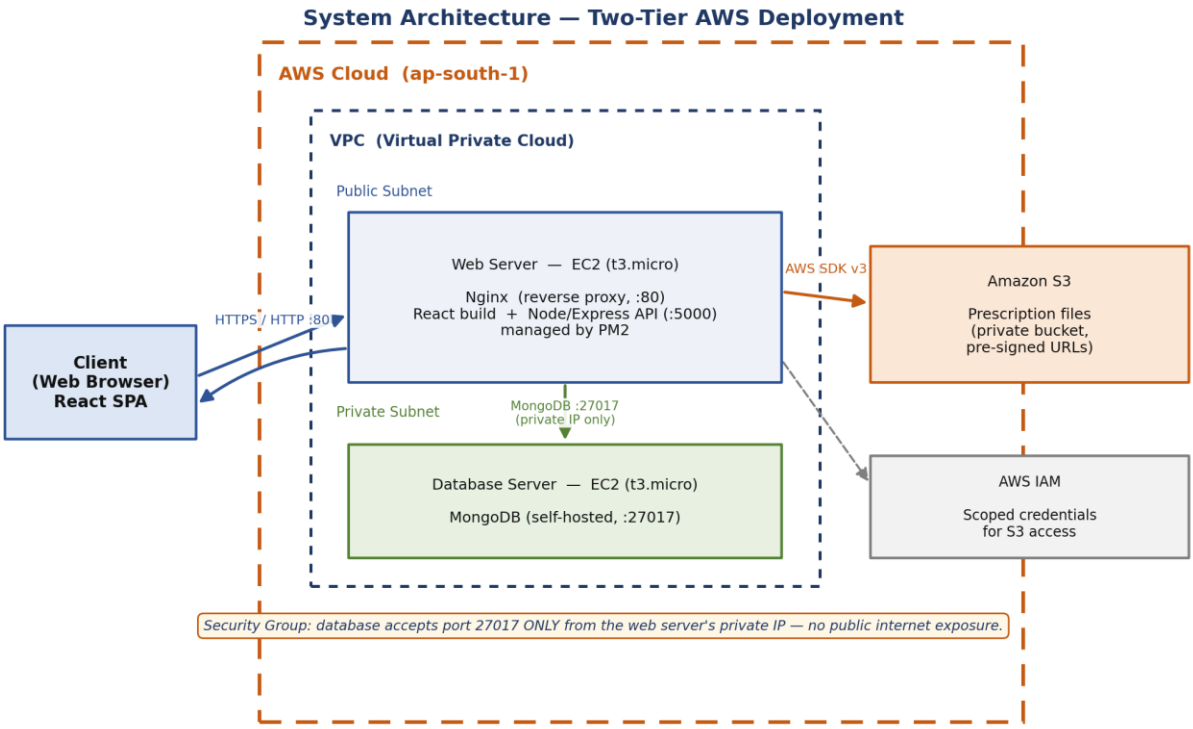
1. Design a role-aware platform that serves two distinct user types – patients and doctors – from a single codebase.
2. Implement stateless authentication using JSON Web Tokens (JWT) and store passwords only in hashed form using bcrypt.
3. Allow patients to search registered doctors by specialisation and view each doctor's experience, fee and available days.
4. Provide slot-based booking that rejects a slot the moment it is taken, preventing two patients from reserving the same time.
5. Give doctors a dashboard to approve, reject, complete or annotate the appointments assigned to them.
6. Enable secure upload of prescription images/PDFs to Amazon S3 and return them through expiring pre-signed URLs.
7. Expose all functionality through a clean, versioned REST API consumed by a React single-page application.
8. Deploy the whole system on AWS with a network design that isolates the database from the public internet.

4. System Architecture & Flowchart

4.1 Two-Tier Cloud Architecture

Rather than running everything on one server, the deployment is split into two Amazon EC2 instances placed inside the same Virtual Private Cloud (VPC). This separation limits the damage an attacker can do and lets each tier be scaled independently.

- **Web Server (public tier) –** hosts the Node.js/Express API under a PM2 process manager and serves the compiled React files through Nginx. Nginx also reverse-proxies every /api request to the backend on port 5000. This instance holds a public IP and accepts HTTP traffic on port 80.
- **Database Server (private tier) –** runs a self-hosted MongoDB Community instance. Its security group allows inbound traffic on port 27017 only from the web server's private IP, so the database has no exposure to the open internet.
- **Object Storage –** a managed Amazon S3 bucket stores uploaded prescription files. The backend writes to it with the AWS SDK and hands clients temporary signed links instead of making the bucket public.



4.2 Request Flow

A typical request travels through the stack as follows: the browser loads the React app from Nginx; the app calls a REST endpoint such as POST /api/appointments; Nginx forwards it to Express; a middleware verifies the JWT; the controller reads or writes MongoDB over the private link; and, for file operations, the backend talks to S3. The textual flowchart below captures the end-to-end journey of a booking.

Stage	What happens
1. Access	Patient/doctor registers or logs in → server issues a signed JWT.
2. Search	Patient queries doctors by specialisation via GET /api/doctors.
3. Select	System fetches the chosen doctor's free slots for the picked date.
4. Book	POST /api/appointments checks for conflicts and stores the booking as 'pending'.
5. Manage	Doctor approves / rejects / completes the request from the dashboard.
6. Prescribe	Doctor uploads a prescription file → stored in Amazon S3.
7. Review	Patient reopens history and views prescriptions through a signed URL.

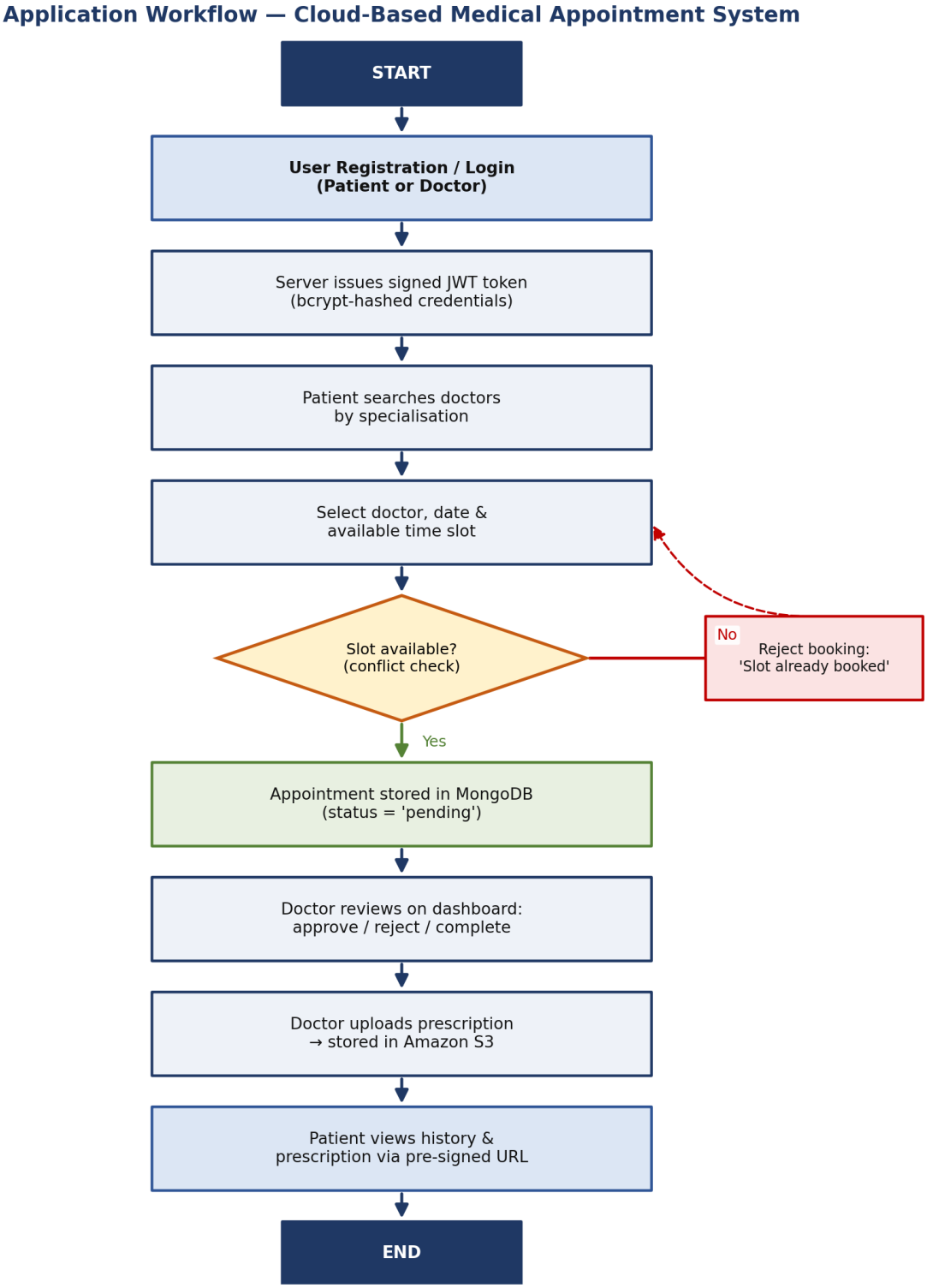


Figure 1: End-to-end workflow of the Cloud-Based Medical Appointment System.

5. Implementation Details

The backend follows a layered structure – routes delegate to controllers, controllers use Mongoose models, and cross-cutting concerns (authentication, validation, error handling, file upload) live in dedicated middleware. Each module is described below.

5.1 Authentication & Authorisation

Registration accepts a name, email, password and role. The password is never stored in plain text: a Mongoose pre-save hook runs bcrypt with a salt factor of 12 before the document is written, and the password field is excluded from queries by default. On successful login the server signs a JWT containing the user id with a 30-day expiry. Every protected route runs an auth middleware that reads the Bearer token, verifies its signature, loads the matching user and attaches it to the request. Because the token is self-contained, the server keeps no session state, which is what allows the API to scale horizontally.

5.2 Doctor Discovery

When a user registers as a doctor, a linked Doctor profile is created holding professional attributes – specialisation, years of experience, qualification, consultation fee, hospital and weekly available slots. Patients browse these profiles through the doctors endpoint, which supports filtering by specialisation so a patient can, for example, list only cardiologists.

5.3 Conflict-Free Appointment Booking

Booking is the most safety-critical operation, so the controller performs two independent conflict checks before persisting anything. First it confirms that the requested doctor, date and time slot are not already reserved by another patient in a pending or approved state. Second it confirms the requesting patient does not already hold an appointment in that same slot. Only if both checks pass is the record created with an initial status of 'pending'. The date is normalised to the full UTC day so that a slot comparison is not thrown off by time components, and compound indexes on (doctorId, date, timeSlot) keep these look-ups fast.

5.4 Appointment Lifecycle

An appointment moves through a controlled set of states – pending, approved, rejected, completed and cancelled. The rules for who may trigger which transition are enforced on the server, not merely in the UI:

- a patient may only cancel their own booking, and only while it is still pending or approved;
- a doctor may only approve, reject or complete appointments that belong to their own profile;
- once an appointment is completed or cancelled it is locked against further changes.

Every read endpoint also checks ownership, so one user can never view or modify another user’s appointment even by guessing its id.

5.5 Cloud Prescription Uploads

Doctors attach a prescription to a completed consultation. The file is received in memory by Multer, then streamed to Amazon S3 through the AWS SDK v3 with a randomly generated UUID key so filenames never collide. The bucket itself is kept private; when a patient later opens a prescription, the backend generates a pre-signed URL that grants read access for one hour and then expires. This gives the durability of cloud storage without ever exposing the bucket publicly.

5.6 Frontend (React SPA)

The client is a Vite-powered React single-page application. Routing is handled by React Router, authentication state lives in a React Context that persists the JWT, and a ProtectedRoute component guards pages that require a login. A central Axios service attaches the token to every request and centralises error handling, while toast notifications give the user immediate feedback. Dedicated pages exist for doctor search, booking, the patient and doctor dashboards, appointment history and profile management.

6. Database Design & API Endpoints

6.1 Data Model

The User collection is common to both roles; a Doctor document extends a user with professional details; Appointment and Prescription documents reference users and doctors by ObjectId.

Collection	Key Fields	Purpose
User	name, email (unique), password (hashed), role, phone	Login identity for patients and doctors.
Doctor	userId, specialisation, experience, qualification, fee, availableSlots	Professional profile linked 1-to-1 with a user.
Appointment	patientId, doctorId, date, timeSlot, reason, status, notes	A booking and its lifecycle state.
Prescription	appointmentId, doctorId, patientId, fileKey, fileUrl	Pointer to the file stored in Amazon S3.

Figure 2: Core collections and their relationships.

6.2 REST API Surface

All resources are exposed under a common /api prefix and follow REST conventions.

Method & Path	Access	Description
POST /api/auth/register	Public	Create a patient or doctor account.
POST /api/auth/login	Public	Authenticate and receive a JWT.
GET /api/auth/profile	Private	Fetch the logged-in user's profile.
GET /api/doctors	Public	List / filter doctors by specialisation.
POST /api/appointments	Patient	Book a conflict-checked appointment.
GET /api/appointments	Private	List appointments for the current user.
PATCH /api/appointments/:id	Private	Approve / reject / complete / cancel.
GET /api/appointments/history	Private	Fetch past/closed appointments.
POST /api/prescriptions	Doctor	Upload a prescription file to S3.
GET /api/health	Public	Service health probe.

Figure 3: Summary of the REST API endpoints.

7. Cloud Deployment & Security

7.1 Provisioning the Infrastructure

Two t3.micro Ubuntu 24.04 instances were launched in the ap-south-1 (Mumbai) region. Provisioning was scripted with the AWS CLI so the environment is reproducible: the script reads the running web server's VPC, subnet and private IP from the instance metadata service, creates a dedicated security group for the database, and opens port 27017 exclusively to the web server's private address. The database instance is then launched into the same subnet and its private IP is recorded for the backend connection string.

7.2 Web Tier Configuration

On the web server, Node.js 20, Nginx and PM2 are installed by a setup script. The React client is built for production and its static files are copied into the Nginx web root. Nginx serves the single-page app and proxies /api/ traffic to the Node process on port 5000, while PM2 keeps the backend alive across crashes and reboots.

7.3 Security Measures

- **Network isolation –** The database accepts connections only from the web server's private IP; it is invisible to the public internet.
- **Credential hashing –** Passwords are hashed with bcrypt (cost 12); the raw password is never stored or logged.
- **Token authentication –** Every protected endpoint requires a valid signed JWT, and controllers additionally verify resource ownership.
- **Scoped file access –** The S3 bucket is private; files are reached only through pre-signed URLs that expire after one hour.
- **Input validation –** express-validator sanitises and validates request bodies to reject malformed or malicious input.

7.4 AWS Services Used

AWS Service	Role in the Project
EC2 (Web)	Runs the Node/Express API and serves the React build via Nginx.
EC2 (Database)	Hosts a private, self-managed MongoDB instance.
S3	Durable object storage for prescription files.
IAM	Programmatic credentials scoped to the S3 bucket for the backend.
VPC / Security Groups	Network isolation between the public and private tiers.

8. Tools & Technologies Used

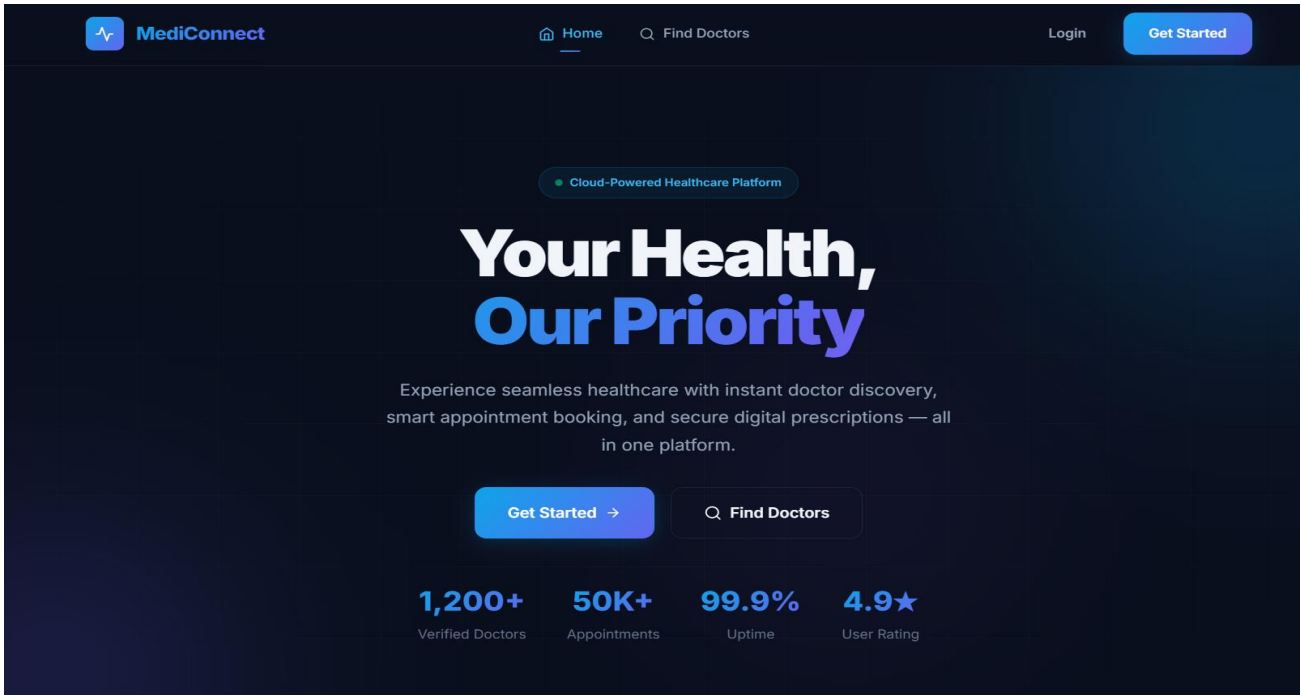
Layer	Technologies
Frontend	React 18, Vite, React Router DOM, Axios, React Hot Toast, Vanilla CSS
Backend	Node.js 20, Express.js, Mongoose ODM, JWT, bcryptjs, Multer, express-validator
Database	MongoDB Community Edition (self-hosted on EC2)
Cloud	AWS EC2, AWS S3, AWS IAM, VPC & Security Groups
Server Ops	Nginx (reverse proxy), PM2 (process manager), AWS CLI
Tooling	Visual Studio Code, Git & GitHub, Postman

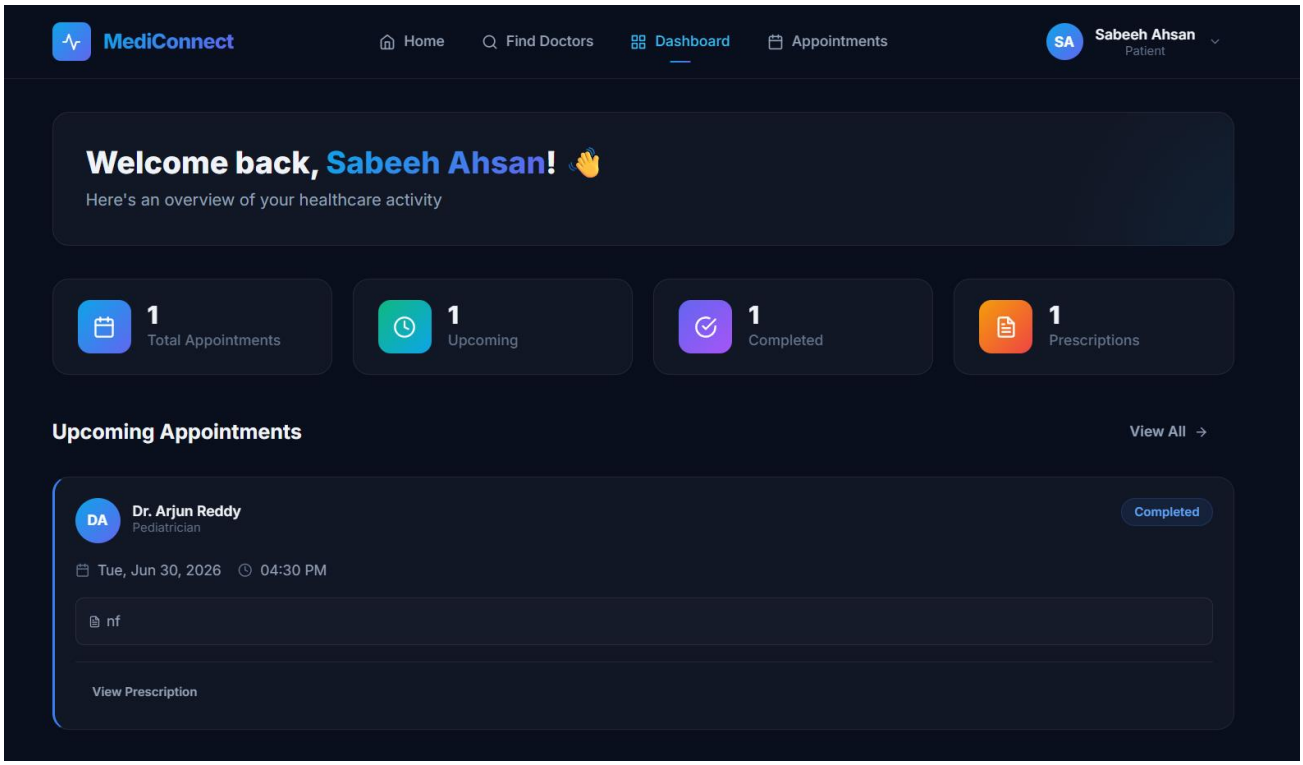
Figure 4: Technology stack grouped by layer.

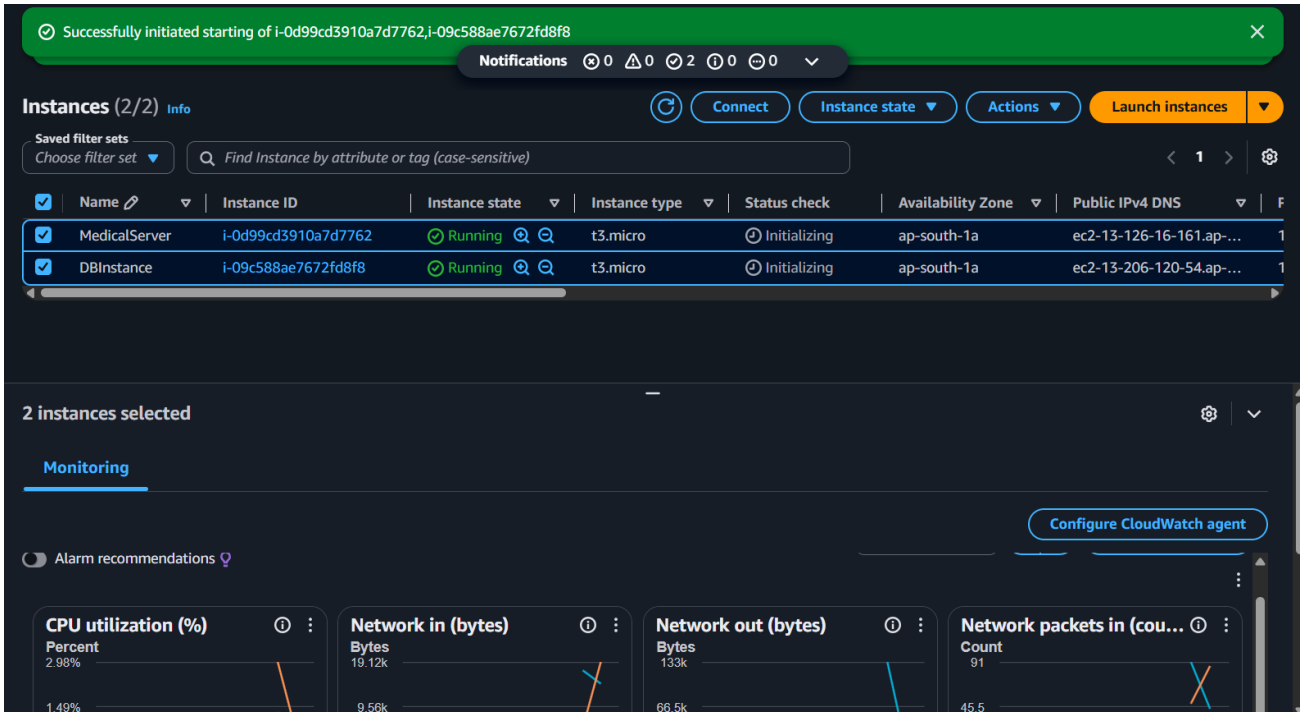
9. Results & Outcomes

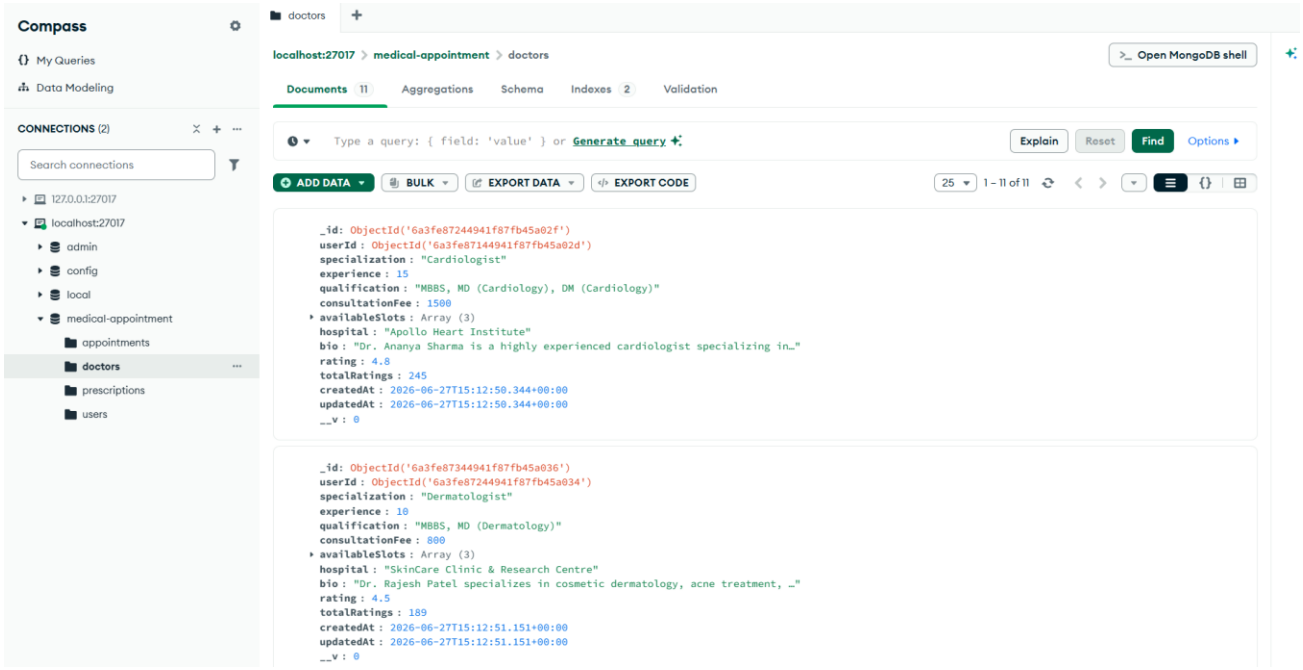
The completed system meets each objective set out at the start:

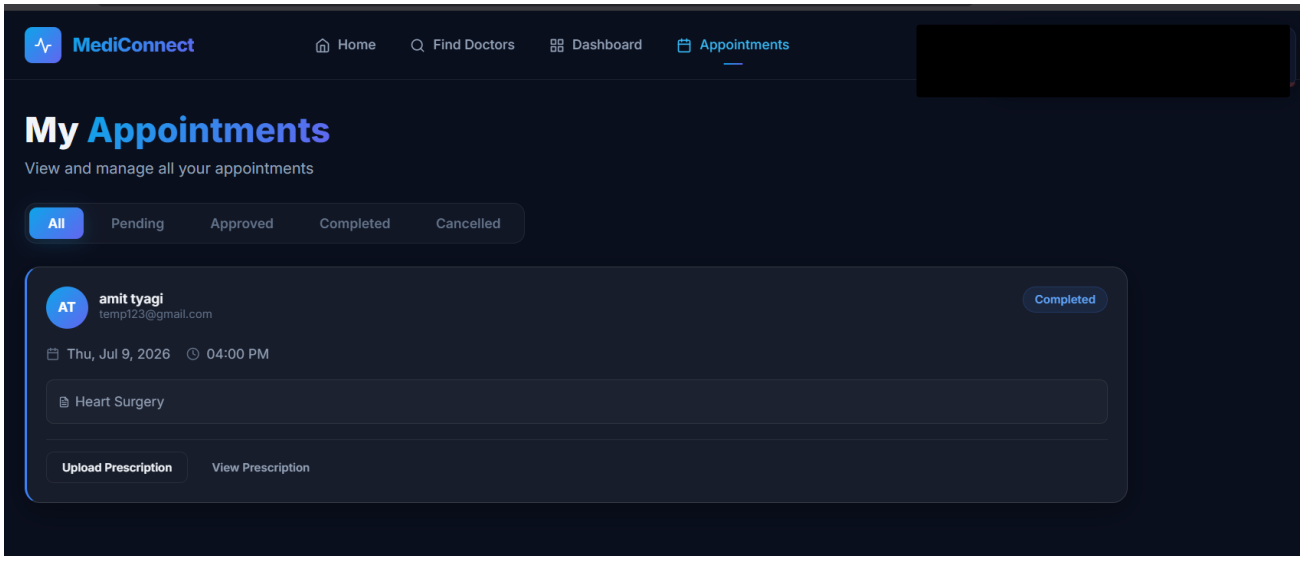
- Patients and doctors can register, log in and stay authenticated across sessions through JWT tokens.
- Doctors are searchable by specialisation, and their availability, fee and experience are shown before booking.
- The double-booking guard was verified by attempting to reserve an already-taken slot; the API correctly rejects it with a clear message.
- Doctors can drive an appointment through its full lifecycle from the dashboard, and patients see the updated status immediately.
- Prescriptions upload successfully to Amazon S3 and open through time-limited signed links, confirming the storage integration.
- The application is reachable over the public internet via the web server’s address, while the database remains unreachable from outside the VPC – confirmed by a failed external connection test on port 27017.

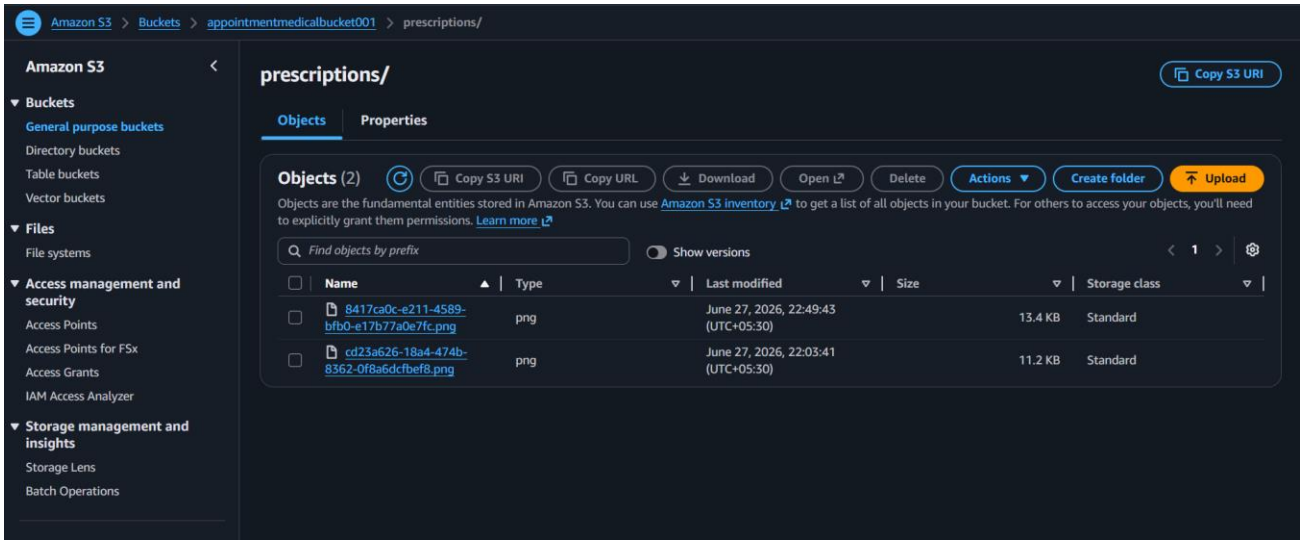

MediConnect
Home
Find Doctors
Login
Get Started
Cloud-Powered Healthcare Platform
Your Health,
Our Priority
Experience seamless healthcare with instant doctor discovery, smart appointment booking, and secure digital prescriptions — all in one platform.
Get Started →
Find Doctors
1,200+
Verified Doctors
50K+
Appointments
99.9%
Uptime
4.9★
User Rating


MediConnect
Home
Find Doctors
Dashboard
Appointments
SA
Sabeeh Ahsan
Patient
Welcome back, Sabeeh Ahsan! 👋
Here's an overview of your healthcare activity
1
Total Appointments
1
Upcoming
1
Completed
1
Prescriptions
Upcoming Appointments
View All →
DA
Dr. Arjun Reddy
Pediatrician
Completed
Tue, Jun 30, 2026
04:30 PM
nf
View Prescription


Successfully initiated starting of i-0d99cd3910a7d7762,i-09c588ae7672fd8f8
Notifications 0 0 2 0 0
Instances (2/2) Info
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Find Instance by attribute or tag (case-sensitive)
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4 DNS
MedicalServer
i-0d99cd3910a7d7762
Running
t3.micro
Initializing
ap-south-1a
ec2-13-126-16-161.ap-...
DBInstance
i-09c588ae7672fd8f8
Running
t3.micro
Initializing
ap-south-1a
ec2-13-206-120-54.ap-...
2 instances selected
Monitoring
Configure CloudWatch agent
Alarm recommendations
CPU utilization (%)
Percent
2.98%
1.49%
Network in (bytes)
Bytes
19.12k
9.56k
Network out (bytes)
Bytes
133k
66.5k
Network packets in (cou...
Count
91
45.5


Compass
My Queries
Data Modeling
CONNECTIONS (2)
Search connections
127.0.0.1:27017
localhost:27017
admin
config
local
medical-appointment
appointments
doctors
prescriptions
users
doctors
localhost:27017 > medical-appointment > doctors
Open MongoDB shell
Documents 11
Aggregations
Schema
Indexes 2
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
BULK
EXPORT DATA
EXPORT CODE
25
1 - 11 of 11
_id: ObjectId('6a3fe87244941f87fb45a02f')
userId : ObjectId('6a3fe87144941f87fb45a02d')
specialization : "Cardiologist"
experience : 15
qualification : "MBBS, MD (Cardiology), DM (Cardiology)"
consultationFee : 1500
availableSlots : Array (3)
hospital : "Apollo Heart Institute"
bio : "Dr. Ananya Sharma is a highly experienced cardiologist specializing in…"
rating : 4.8
totalRatings : 245
createdAt : 2026-06-27T15:12:50.344+00:00
updatedAt : 2026-06-27T15:12:50.344+00:00
__v : 0
_id: ObjectId('6a3fe87344941f87fb45a036')
userId : ObjectId('6a3fe87244941f87fb45a034')
specialization : "Dermatologist"
experience : 10
qualification : "MBBS, MD (Dermatology)"
consultationFee : 800
availableSlots : Array (3)
hospital : "SkinCare Clinic & Research Centre"
bio : "Dr. Rajesh Patel specializes in cosmetic dermatology, acne treatment, …"
rating : 4.5
totalRatings : 189
createdAt : 2026-06-27T15:12:51.151+00:00
updatedAt : 2026-06-27T15:12:51.151+00:00
__v : 0


MediConnect
Home
Find Doctors
Dashboard
Appointments
My Appointments
View and manage all your appointments
All
Pending
Approved
Completed
Cancelled
AT
amit tyagi
temp123@gmail.com
Completed
Thu, Jul 9, 2026
04:00 PM
Heart Surgery
Upload Prescription
View Prescription


Amazon S3
Buckets
appointmentmedicalbucket001
prescriptions/
Amazon S3
Buckets
General purpose buckets
Directory buckets
Table buckets
Vector buckets
Files
File systems
Access management and security
Access Points
Access Points for FSx
Access Grants
IAM Access Analyzer
Storage management and insights
Storage Lens
Batch Operations
prescriptions/
Copy S3 URI
Objects
Properties
Objects (2)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
Show versions
Name
Type
Last modified
Size
Storage class
8417ca0c-e211-4589-bfb0-e17b77a0e7fc.png
png
June 27, 2026, 22:49:43 (UTC+05:30)
13.4 KB
Standard
cd23a626-18a4-474b-8362-0f8a6dcfbef8.png
png
June 27, 2026, 22:03:41 (UTC+05:30)
11.2 KB
Standard

10. Conclusion & Future Scope

10.1 Conclusion

MediConnect demonstrates how a routine clinical process can be moved onto the cloud in a way that is secure, reliable and accessible from anywhere. By combining a MERN application with a deliberately isolated two-tier AWS deployment, the project shows the practical value of separating a public web tier from a private data tier and of offloading heavy files to object storage. The result is a working platform where appointments cannot collide, records are never lost, and access to every resource is authenticated and authorised. Beyond the feature set, the project was a hands-on exercise in cloud provisioning, network security groups, reverse proxying and stateless API design.

10.2 Future Enhancements

9. Real-time notifications through WebSockets or AWS SNS to alert users about appointment status changes by email or SMS.
10. Integrated video consultations using WebRTC for remote telemedicine.
11. An online payment gateway to collect consultation fees at the time of booking.
12. Auto-scaling by placing the web tier behind an Application Load Balancer with an Auto Scaling Group to absorb traffic spikes.
13. Automated database backups and a managed database option (such as MongoDB Atlas or Amazon DocumentDB) for higher availability.
14. A mobile application sharing the same REST API for on-the-go access.

11. References

15. React Documentation – https://react.dev
16. Node.js Documentation – https://nodejs.org
17. Express.js Documentation – https://expressjs.com
18. MongoDB Documentation – https://www.mongodb.com/docs
19. Mongoose ODM Documentation – https://mongoosejs.com/docs
20. Amazon EC2 Documentation – https://docs.aws.amazon.com/ec2
21. Amazon S3 Documentation – https://docs.aws.amazon.com/s3
22. AWS SDK for JavaScript v3 – https://docs.aws.amazon.com/sdk-for-javascript
23. JSON Web Tokens – https://jwt.io
24. Nginx Documentation – https://nginx.org/en/docs
25. PM2 Process Manager – https://pm2.keymetrics.io
26. REST API Design Guide – https://restfulapi.net